

Performance Testing

Date	10 July 2026
Team ID	xxxxxx
Project Name	Raising Waters
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman / Python custom benchmarking scripts
Type of Testing	Load and Latency Testing
Target Module / API	endpoint (Flask API Route)
Test Environment	Localhost Server Environment
Test Date	10 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Baseline single request check	1	1	Instant ML model classification output
2	Standard coordinator dashboard refresh	10	30	Response time remains under 2 seconds
3	Peak monsoon automated data feed surge	50	60	Concurrent requests process without data loss
4				

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.45s	pass	Fast inference processing.
2	Response Time (Max)	< 5 seconds	1.20s		Well within safety parameters
3	Throughput (Req/sec)		25 req/s		Handles local load easily.
4	Error Rate	< 1%	0.0%		No dropped requests.
5	CPU Utilization	< 80%	24%		Minimal load on host system
6	Memory Utilization	< 80%	35%		Lightweight execution.

Step 4: Observations & Analysis

Key Findings:

[Describe what the test results revealed about your system's performance]

The system operates efficiently under normal and simulated peak conditions. The pre-trained machine learning pipeline reads .pkl model weights instantly with minimal computing overhead.

Bottlenecks Identified:

[List any performance bottlenecks observed during testing]

: Occasional network wait delays when making continuous sequential calls to the external live weather data API endpoint.

Optimization Steps Taken:

[Describe any changes or optimizations made to improve performance]

Optimized the inference route by handling API connection timeouts cleanly and caching the latest response data payload locally to save processing cycles.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

[Insert Screenshot 1 here]

